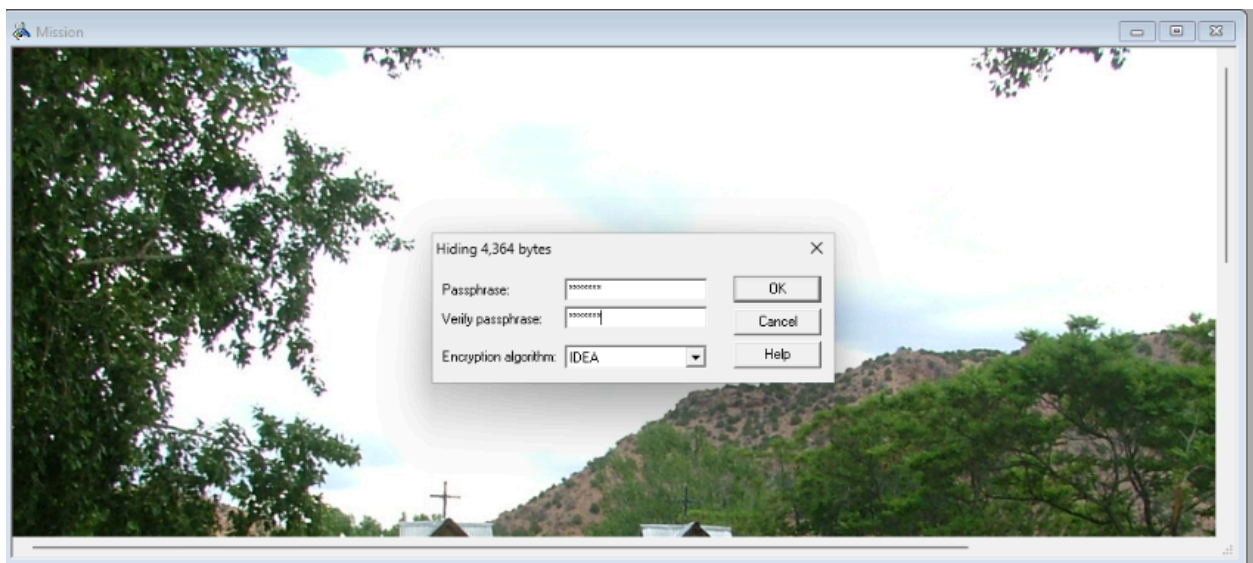
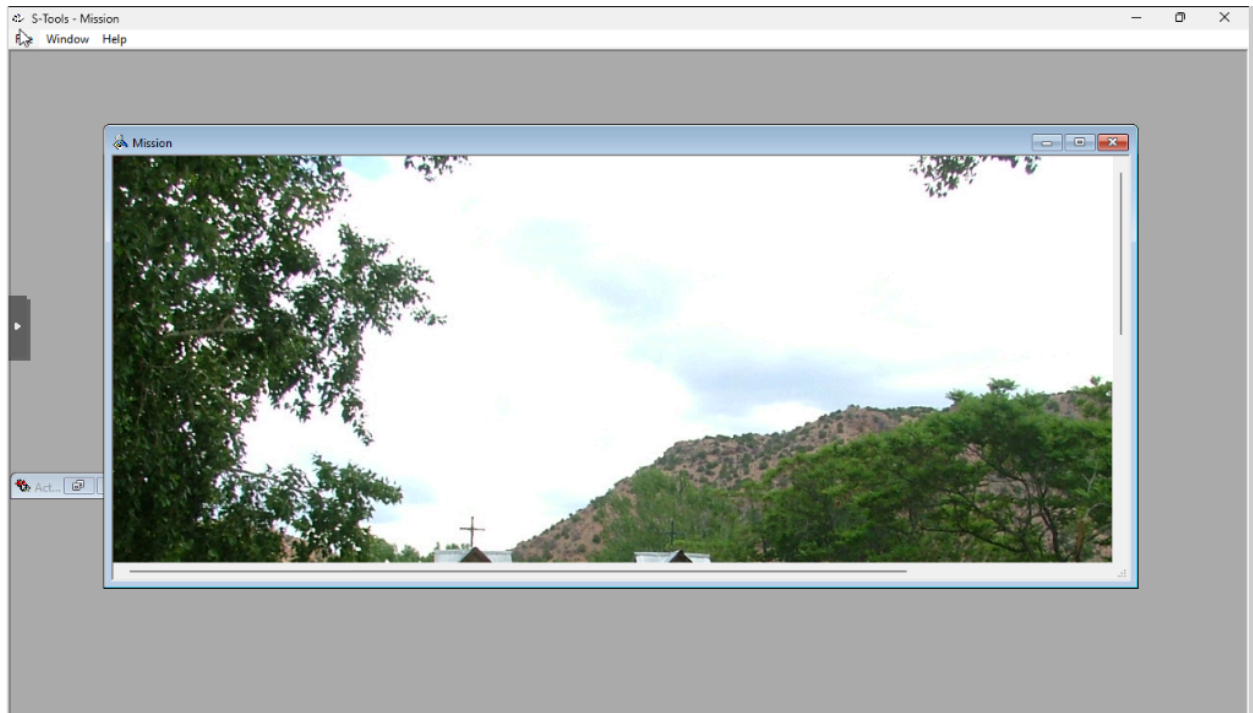
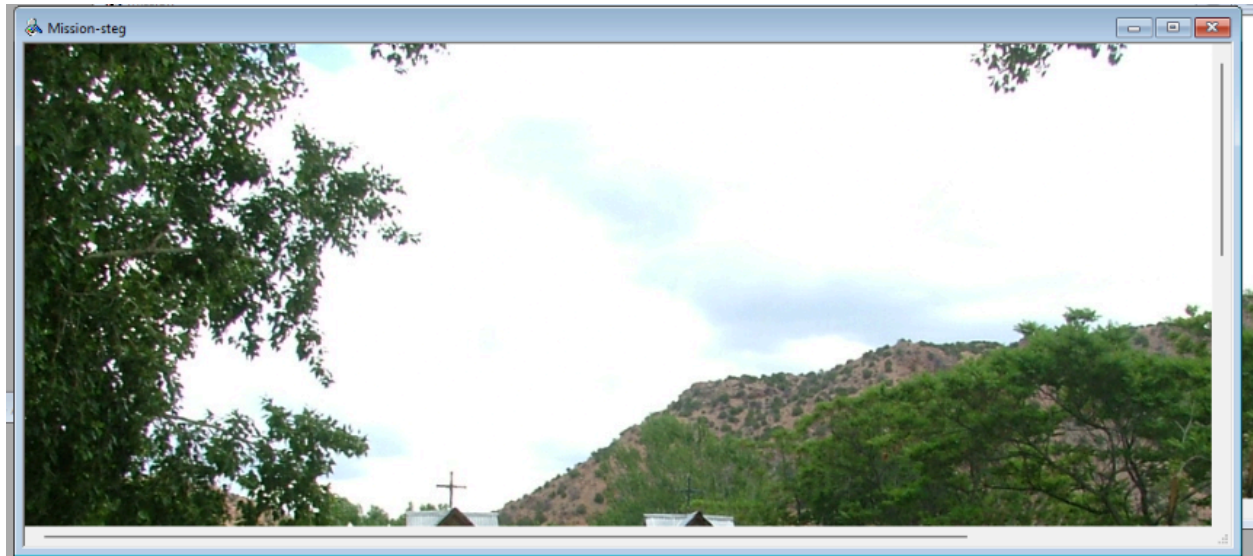
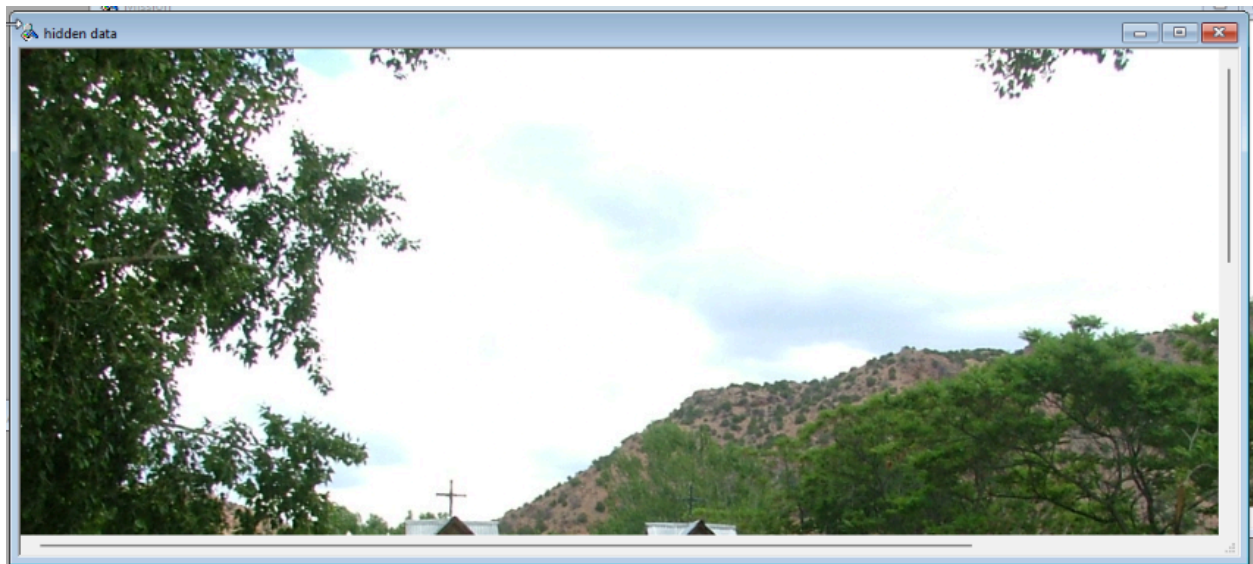


8-5 Hands on Project

Jack Irvin

10/15/2025





```
C:\Users\Student\Desktop\Work\C08H0P805>
```

```
C:\Users\Student\Desktop\Work\C08H0P805>comp Mission.bmp Mission-steg.bmp > Mission-compare.txt
Compare more files (Y/N) ? Y
Name of first file to compare: Mission.bmp
Name of second file to compare: Mission-steg.bmp
Option:
Compare more files (Y/N) ? N
C:\Users\Student\Desktop\Work\C08H0P805>
```

```
Comparing Mission.bmp and Mission-steg.bmp...
Compare error at OFFSET 209
file1 = 6B
file2 = 6A
Compare error at OFFSET 392
file1 = 7F
file2 = 7E
Compare error at OFFSET 3C5
file1 = 96
file2 = 97
Compare error at OFFSET 440
file1 = 7F
file2 = 7E
Compare error at OFFSET 604
file1 = 6A
file2 = 6B
Compare error at OFFSET 844
file1 = 8A
file2 = 8B
Compare error at OFFSET 879
file1 = 5D
file2 = 5C
Compare error at OFFSET 92E
file1 = 2C
file2 = 2D
Compare error at OFFSET 965
file1 = A8
file2 = A9
Compare error at OFFSET B38
file1 = A3
file2 = A2
10 mismatches - ending compare
```

In this lab, I worked with two bitmap image files, Mission.bmp and Mission-steg.bmp, to explore how embedding hidden data changes an image at the binary level. The original Mission.bmp was a normal bitmap image used as the host file. I used S-Tools4 to hide a text file named USDECINP.rtf inside it using the passphrase hop08-5. The new file was saved as Mission-steg.bmp. Both images looked completely identical when viewed normally, which shows how steganography hides information without altering how the file appears on the surface.

After creating the steganography file, I compared the two images using the Windows comp command by using command prompt and saved the results in Mission-compare.txt. The comparison showed that there were several mismatches between the original and modified files. These differences appeared at hexadecimal offsets like 209, 392, 440, 604, 844, 879, 92E, 965, and B38. Each mismatch showed small changes in the hexadecimal values, like 6B changing to 6A or 7F changing to 7E. Even though these changes seem tiny, they indicate where the hidden data was embedded into the least significant bits of the image file.

The pattern of mismatches shows that the hidden message wasn't placed in one section of the file but was spread out across different data areas. This technique makes the hidden information harder to detect and preserves the image quality. None of the changes affected how the picture looked, which means the steganographic process worked exactly as intended. Another fascinating thing about the process is that somebody not looking for these changes wouldn't be able to tell an image was altered for steganography.

Overall, this lab showed how effective steganography can be at concealing data. The slight hexadecimal differences proved that hidden information can exist within a file without

altering its visual appearance. By using a passphrase like hop08-5, the data also stays protected, showing how encryption and steganography can work together to hide information securely.